

DESARROLLO DE APLICACIONES BÁSICO · TEMA 12

Manual paso a paso Web_Sesion12

Construcción de un sitio web con acceso a datos mediante Entity Framework Core y LINQ, procedimientos almacenados y AJAX moderno (fetch), sobre la base de datos VentasLeon.

1. Crear el proyecto e instalar EF Core

Construiremos **Web_Sesion12**, un sitio que consulta la base de datos **VentasLeon**. Primero creamos el proyecto y agregamos Entity Framework Core.

Paso 1 En Visual Studio 2026 elija **Crear un proyecto nuevo** y seleccione la plantilla **ASP.NET Core Web App (Razor Pages)**.

Paso 2 Asigne el nombre `Web_Sesion12` a la solución y al proyecto.

Paso 3 Elija el framework **.NET 10**, deje marcada la casilla de HTTPS y haga clic en **Crear**.

Paso 4 Abra **Administrar paquetes NuGet** e instale estos dos paquetes:

```
Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Tools    // habilita los comandos de la consola
```

Recuerde: la plantilla "ASP.NET Core Web App (Razor Pages)" puede aparecer en inglés aunque su Visual Studio esté en español. Es la correcta.

Crear la base de datos

Abra **SQL Server Management Studio** (o Azure Data Studio) y ejecute una sola vez el script `VentasLeon.sql` que acompaña a este manual. Crea la base de datos, las tablas, datos de ejemplo y los procedimientos almacenados que usaremos.

2. Generar el modelo y configurar la conexión

En un proyecto real, el modelo se genera desde la base de datos con un solo comando en la **Consola del Administrador de paquetes** (database-first):

```
Scaffold-DbContext
"Server=.;Database=VentasLeon;Trusted_Connection=True;TrustServerCertificate=True"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Modelos -Context
VentasLeonContext
```

Esto crea una clase por cada tabla y el `VentasLeonContext`. En este manual las clases ya vienen escritas en la carpeta **Modelos** para enfocarnos en las consultas.

La cadena de conexión en appsettings.json

A diferencia del material clásico, la cadena ya no va en Web.config sino en `appsettings.json`:

```
{
  "ConnectionStrings": {
    "VentasLeon":
"Server=.;Database=VentasLeon;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

Registrar el DbContext en Program.cs

Finalmente registramos el contexto y la capa de datos en el contenedor de dependencias:

```
builder.Services.AddRazorPages();

// se registra el DbContext con la cadena de conexion de appsettings.json
builder.Services.AddDbContext<VentasLeonContext>(op =>
    op.UseSqlServer(builder.Configuration.GetConnectionString("VentasLeon")));

// la capa de datos: expone los metodos de consulta
builder.Services.AddScoped<VentasRepositorio>();
```

Inyección de dependencias: desde aquí las páginas reciben la capa de datos lista en su constructor; no se crea nada con `new`.

3. Ejemplo 1 · Consulta con LINQ

La capa de datos (`Datos/VentasRepositorio.cs`) concentra el acceso a la base. Las páginas nunca consultan directamente. Este método calcula la deuda del cliente con LINQ:

```
// consulta a la base de datos via LINQ to Entities (join + Sum)
public async Task<decimal> CalcularDeudaClienteLinqAsync(string codigo)
{
    return await (from f in _ctx.Facturas
                  join d in _ctx.DetalleFacturas on f.NumFac equals d.NumFac
                  where f.CodCli == codigo && f.EstFac == 1
                  select (decimal)(d.CanVen * d.PreVen)).SumAsync();
}
```

Y este lista las facturas del cliente entre dos fechas (sintaxis de método). `Include` trae el detalle para calcular el total de cada factura:

```
public async Task<List<Factura>> ListarFacturasLinqAsync(string codigo, DateTime ini,
DateTime fin)
{
    return await _ctx.Facturas
        .Include(f => f.Detalles)
        .Where(f => f.CodCli == codigo && f.FecFac >= ini && f.FecFac <= fin)
        .OrderBy(f => f.FecFac)
        .ToListAsync();
}
```

La página `Consulta.cshtml.cs` solo invoca esos métodos en su `OnPost` y la vista muestra la tabla. Pruebe con el cliente `C001`.

Idea clave: es prácticamente el mismo LINQ del material clásico; cambia que el contexto se inyecta (`_ctx`) y que usamos versiones asíncronas (`SumAsync` , `ToListAsync`).

4. Ejemplo 2 · Procedimientos almacenados

A veces la lógica ya vive en la base de datos. EF Core puede ejecutar procedimientos almacenados. Para recibir el conjunto de filas usamos una **entidad sin clave** (keyless), declarada en el contexto:

```
// en VentasLeonContext.OnModelCreating
modelBuilder.Entity<FacturaResumen>().HasNoKey();
```

El método que ejecuta el SP que devuelve filas:

```
// la interpolacion se convierte en parametros: es segura frente a inyeccion SQL
return await _ctx.FacturasResumen
    .FromSql($"EXEC usp_ListarFacturasClienteFechas {codigo}, {ini}, {fin}")
    .ToListAsync();
```

Para un SP con **parámetro de salida** (la deuda) se usa `ExecuteSqlRaw` con un parámetro marcado como `Output`:

```
var pDeuda = new SqlParameter("@vdeuda", SqlDbType.Decimal)
    { Precision = 12, Scale = 2, Direction = ParameterDirection.Output };

await _ctx.Database.ExecuteSqlRawAsync(
    "EXEC usp_DeudaCliente @codigo, @vdeuda OUTPUT",
    new SqlParameter("@codigo", codigo), pDeuda);

return (decimal)pDeuda.Value;
```

Importante: el SP `usp_DeudaCliente` y `usp_ListarFacturasClienteFechas` ya están creados por el script `VentasLeon.sql`. Si no lo ejecutó, este ejemplo fallará.

5. Ejemplo 3 · AJAX moderno (sin recargar)

Aquí reemplazamos el viejo AjaxControlToolkit por AJAX estándar. En el code-behind creamos un **handler** que devuelve JSON. Su nombre empieza por `OnGet` y termina en `Async`:

```
// handler AJAX: devuelve el cliente y su deuda en JSON, sin recargar la pagina
public async Task<IActionResult> OnGetBuscarClienteAsync(string codigo)
{
    var c = await _repo.BuscarClienteAsync(codigo);
    if (c == null) return new JsonResult(new { encontrado = false });

    decimal deuda = await _repo.CalcularDeudaClienteLinqAsync(codigo);
    return new JsonResult(new { encontrado = true, c.RazSocCli, c.RucCli, c.DirCli,
    deuda });
}
```

En la vista, JavaScript llama al handler con `fetch` y actualiza solo el panel. Observe que se invoca por su nombre: `?handler=BuscarCliente`.

```
// pide los datos al servidor y refresca solo el panel
const r = await fetch(`?handler=BuscarCliente&codigo=${codigo}`);
const data = await r.json();
document.getElementById("razon").textContent = data.razSocCli;
document.getElementById("deuda").textContent = formatearSoles(data.deuda);
```

Regla de oro (sigue vigente): el fetch llama al handler, el handler usa la capa de datos (EF Core) y devuelve solo lo necesario. La vista nunca toca la base de datos.

La misma página incluye un selector de fecha (`<input type="date">` , reemplazo del CalendarExtender), un monto con máscara de soles (reemplazo del MaskedEdit) y una confirmación con `confirm()` antes de registrar.

6. Probar la aplicación

Paso 1 Verifique que ejecutó `VentasLeon.sql` y que la cadena de conexión de `appsettings.json` apunta a su servidor.

Paso 2 Presione **F5**. Se abre la página de inicio con los tres ejemplos.

Paso 3 En **1 · Consulta LINQ** escriba el cliente `C001` y consulte: verá su deuda y sus facturas.

Paso 4 En **2 · Procedimiento** repita con `C001`: el resultado es el mismo, pero calculado por los procedimientos almacenados.

Paso 5 En **3 · AJAX** escriba `C001` y pulse "Buscar": el panel se actualiza sin recargar la página.

Clientes de prueba: `C001`, `C002` y `C003`. El cliente `C001` tiene facturas pendientes y canceladas, ideal para ver la deuda.

Si algo falla: el error más común es que la BD no existe o la cadena de conexión no coincide con su instancia de SQL Server (por ejemplo, use `Server=.\SQLEXPRESS` si esa es su instancia).

Con esto cubrimos los tres pilares de la sesión: consultas con LINQ, procedimientos almacenados y AJAX, todo sobre Entity Framework Core.